

CS4423 - Networks

Prof. Götz Pfeiffer

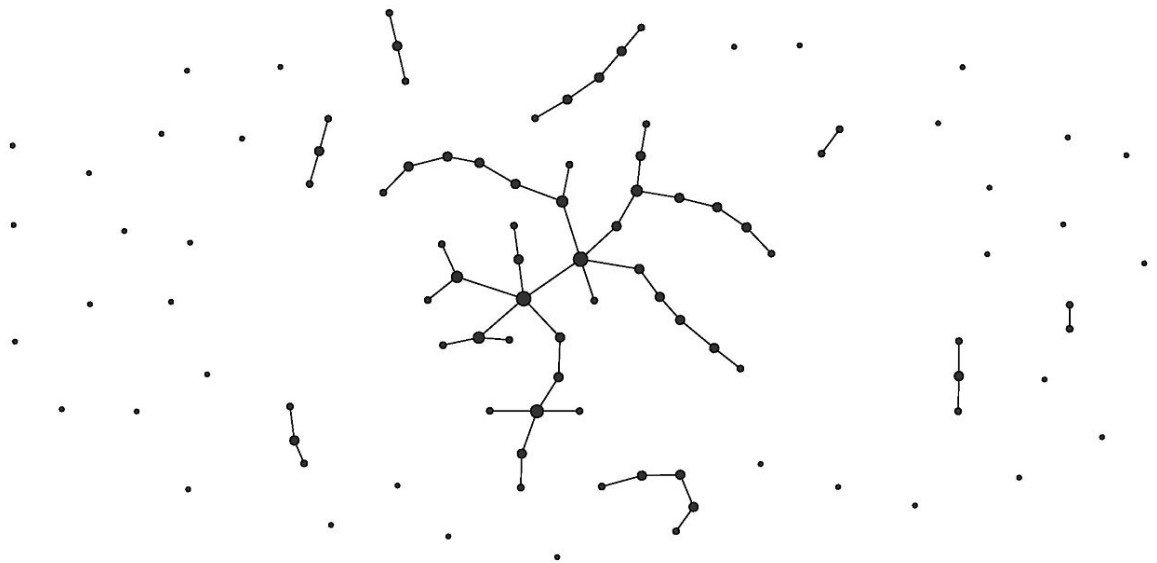
School of Mathematics, Statistics and Applied Mathematics

NUI Galway

3. Random Networks

Lecture 10: Erdős-Rényi Random Graph Models.

A [random graph](https://en.wikipedia.org/wiki/Random_graph) (https://en.wikipedia.org/wiki/Random_graph) is a mathematical model of a family of networks, where certain parameters (like the number of nodes and edges) have fixed values, but other aspects (like the actual edges) are randomly assigned. The simplest example of a random graph is in fact the network $G(n, m)$ with fixed numbers n of nodes and m of edges, placed randomly between the vertices. Although a random graph is not a specific object, many of its properties can be described precisely, in the form of expected values, or probability distributions.



```
In [1]: import networkx as nx
import random
opts = { "with_labels": True, "node_color": 'y' }
opts1 = { "with_labels": True, "node_color": 'lightblue' }
```

Random Samples

In order to randomly select edges on a given vertex set X , we need a procedure for selecting m

from n things randomly, in such a way that each of the $\binom{n}{m}$ subsets of the n things is an equally likely outcome. We first discuss sampling m values in the range $\{0, 1, \dots, n-1\}$.

Maybe the most obvious approach is to select each number in the desired range with probability $p = m/n$. Python's basic random number generator `random.random` returns a random number in the (half-open) interval $[0, 1)$, every time it is called.

Looping with `a` over `range(n)`, if this random number is less than p , we include the current value of `a`, if not we don't.

```
In [2]: def random_sample_B(n, p):
        "sample elements in range(n) with probability p"
        sample = []
        for a in range(n):
            if random.random() < p:
                sample.append(a)
        return sample
```

Let's see a small example:

```
In [3]: n = 100
        m = 10
```

```
In [4]: pick = random_sample_B(n, m/n)
        pick
```

```
Out[4]: [3, 11, 15, 24, 59, 67, 70, 80, 82, 86]
```

```
In [5]: len(pick)
```

```
Out[5]: 10
```

Unfortunately, this gives a sample of size m only on average. Let's check the average size of, say, 10000 samples.

```
In [6]: c = 10000
        sum(len(random_sample_B(n, m/n)) for i in range(c))/c
```

```
Out[6]: 10.0088
```

- Let k be the sample size, as a random variable.
- **Expected value:** $E[k] = np = m$.

Proof: This is a [binomial distribution](https://en.wikipedia.org/wiki/Binomial_distribution) (https://en.wikipedia.org/wiki/Binomial_distribution).

- The probability of a specific subset of size k to be chosen is $p^k(1-p)^{n-k}$.
- As there are $\binom{n}{k}$ subsets of size k , the probability $P(k)$ of the sample to have size k is $P(k) = \binom{n}{k} p^k (1-p)^{n-k}$.

- Thus

$$E[k] = \sum_{k=0}^n P(k) \cdot k = \sum_{k=0}^n k \binom{n}{k} p^k (1-p)^{n-k} = np \sum_{l=0}^{n-1} \binom{n-1}{l} p^l (1-p)^{(n-1)-l} =$$

substituting $l = k - 1$,

- as $k \binom{n}{k} p^k = np \binom{n-1}{k-1} p^{k-1}$ and $(1-p)^{n-k} = (1-p)^{(n-1)-(k-1)}$.

- **Standard deviation:** $\sigma_k = \sqrt{m(1 - m/n)}$.
- $E[k^2] = \sum_{k=0}^n P(k) \cdot k^2 = \dots = n(n-1)p^2 + np$.
- $E[k^2] - E[k]^2 = n(n-1)p^2 + np - (np)^2 = np(1-p)$.

```
In [7]: c = 10000
sum(1 for i in range(c) if len(random_sample_B(n, m/n)) == m)
```

Out[7]: 1294

A simple modification of this procedure will always yield samples of size m [see Knuth: The Art of Computer Programming, Vol. 2, Section 3.4.2, Algorithm S] :

- number a should be selected with probability $(m - c)/(n - a)$, if c items have already been selected.

```
In [8]: def random_sample_A(n, m):
        sample = []
        for a in range(n):
            if (n - a) * random.random() < m - len(sample):
                sample.append(a)
        return sample
```

Let's see a small example.

```
In [9]: pick = random_sample_A(n, m)
pick
```

Out[9]: [1, 16, 18, 34, 45, 66, 72, 75, 93, 97]

```
In [10]: len(pick)
```

Out[10]: 10

We now apply these strategies A and B to the problem of randomly selection edges for a graph on the vertex set $X = \{0, 1, \dots, n-1\}$.

Uniformly Selected Edges

Let us denote by $G(n, m)$ a network with n nodes and m chosen edges, chosen uniformly at

random (out of the possible $\binom{n}{2}$). Equivalently, one can choose uniformly at random one network in the **set** $G(n, m)$ of **all** networks on a given set of n nodes with **exactly** m edges.

Definition (ER Model A: Uniform Random Graphs). Let $n \geq 1$, let $N = \binom{n}{2}$ and let $0 \leq m \leq N$.

The model $G(n, m)$ consists of the ensemble of graphs G with n nodes $X = \{0, 1, \dots, n-1\}$, and m randomly selected edges, chosen uniformly from the N possible edges.

One could think of $G(n, m)$ as a probability distribution $P: G(n, m) \rightarrow \mathbb{R}$, that assigns to each network $G \in G(n, m)$ the same probability

$$P(G) = \binom{N}{m}^{-1},$$

where $N = \binom{n}{2}$.

For example ...

71

3.1 Erdős and Rényi (ER) Models

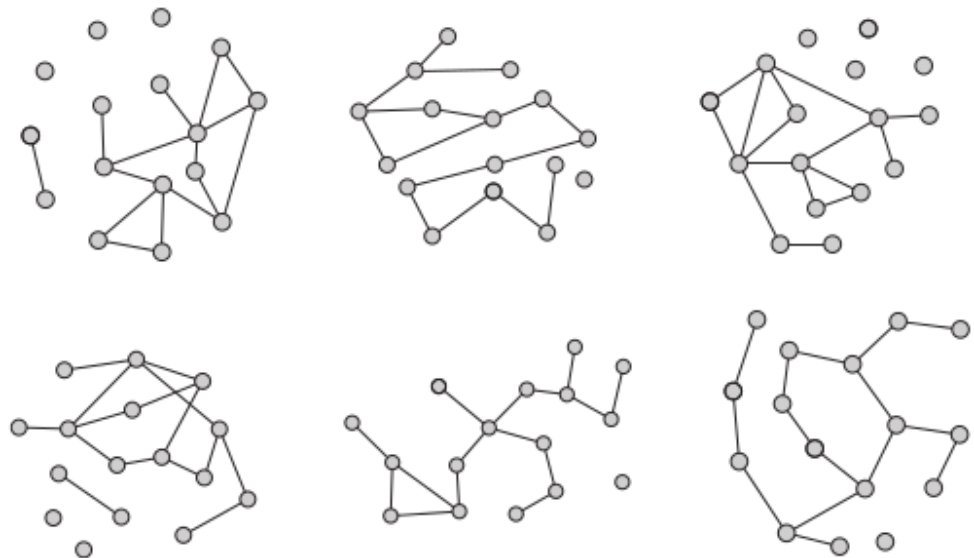


Fig. 3.1 Six different realisations of model A with $N = 16$ and $K = 15$.

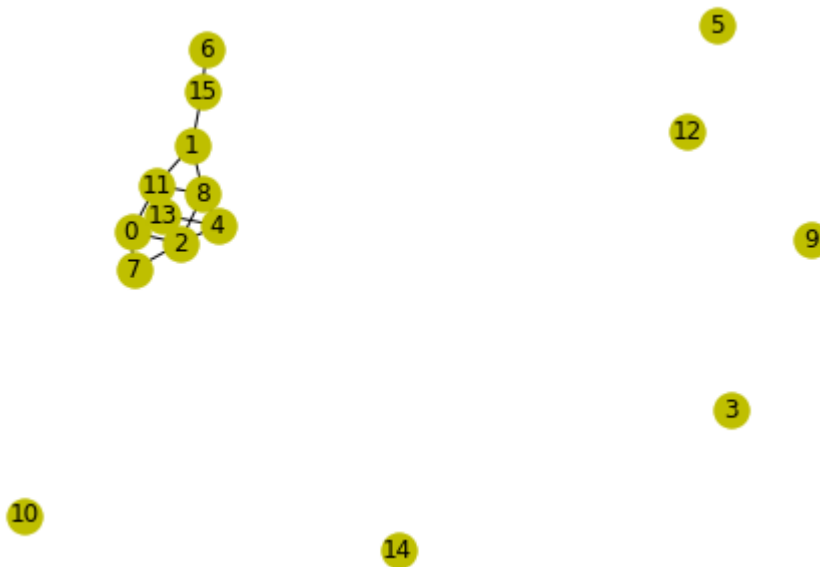
A function `random_graph_A` that takes the order n and the size m of a random ER graph of type A as arguments, and constructs such a graph can simply follow the `random_sample_A` algorithm and sample exactly m of the $N = \binom{n}{2}$ available edges. We introduce explicit counters `a` (for the number of edges that have been considered) and `c` (for the number of chosen edges).

```
In [11]: def random_graph_A(n, m):
        """construct a random type A graph
        with n nodes and m links"""
        a = c = 0
        N = n * (n-1) // 2
        G = nx.empty_graph(n)
        for x in range(n):
            for y in range(x):
                if (N - a) * random.random() < m - c:
                    G.add_edge(x, y)
                    c += 1
            a += 1
        return G
```

Now we can construct and draw a random graph on 16 vertices with 15 edges.

```
In [12]: n = 16
        m = 15
```

```
In [13]: G = random_graph_A(n, m)
        nx.draw(G, **opts)
```

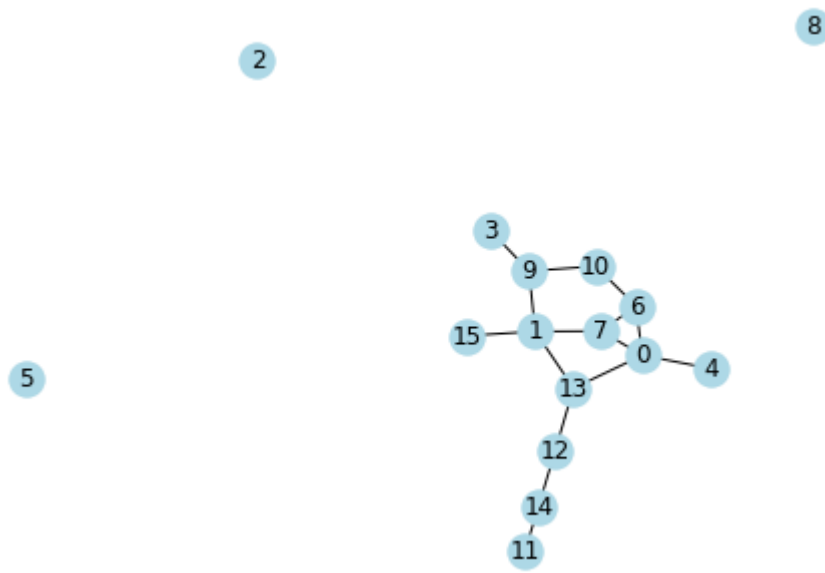


```
In [14]: G.number_of_edges()
```

```
Out[14]: 15
```

The `networkx` version of this random graph constructor is called `gnm_random_graph` and should produce the same random graphs with the same probability.

```
In [15]: G = nx.gnm_random_graph(n, m)
          nx.draw(G, **opts1)
```



```
In [16]: list(G.nodes())
```

```
Out[16]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

An Animated Random Graph

Adding one randomly selected edge at a time ... until the [graph](http://schmidt.nuigalway.ie/cs4423/random.html) (<http://schmidt.nuigalway.ie/cs4423/random.html>) is connected.

Randomly Selected Edges

Definition (ER Model B : Binomial Random Graphs). Let $n \geq 1$, let $N = \binom{n}{2}$ and let $0 \leq p \leq 1$.

The model $G(n, p)$ consists of the ensemble of graphs G with n nodes $X = \{0, 1, \dots, n-1\}$, and each of the N possible edges chosen with probability p .

The probability P_G of a particular graph $G = (X, E)$ with $X = \{0, 1, \dots, n-1\}$ and $m = |E|$ edges in the $G(n, p)$ model is

$$P_G = p^m (1 - p)^{N-m}.$$

For example ...

72

Random Graphs

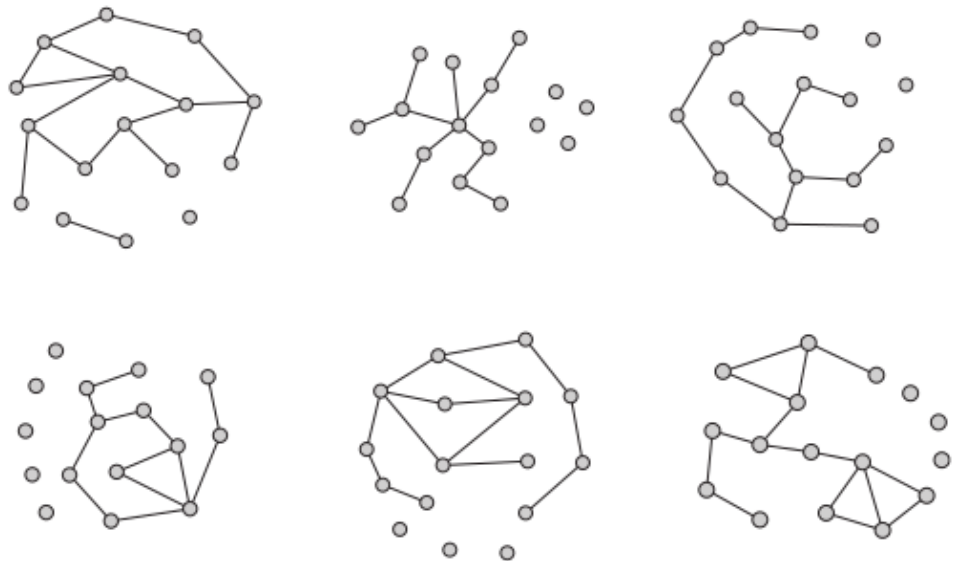


Fig. 3.2 Six different realisations of model B, with $N = 16$ and $p = 0.125$.

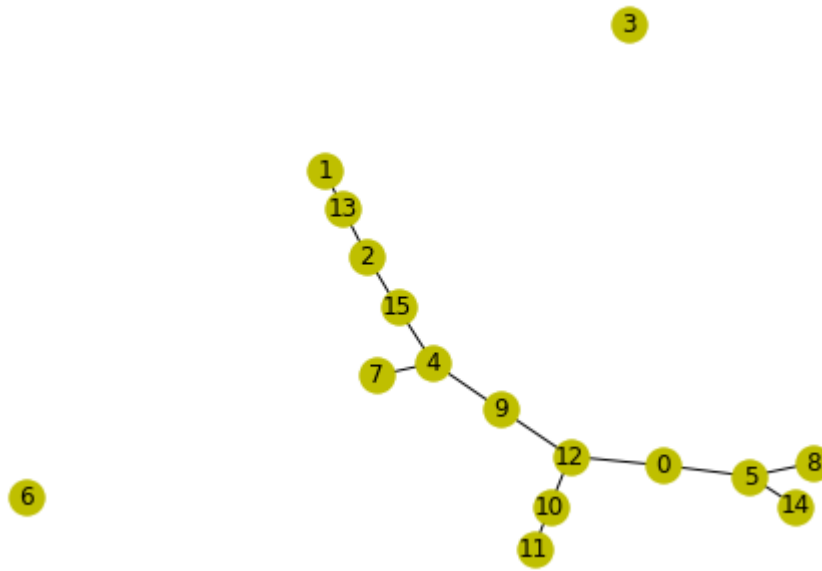
Such a random graph is easy to generate programmatically, following the `random_sample_B` algorithm. python's basic random number generator `random.random` returns a random number in the (half-open) interval $[0, 1)$, every time it is called. If this number is less than p , we include the edge, if not we don't.

```
In [17]: def random_graph_B(n, p):
          """construct a random type B graph
          with n nodes and edge probability p"""
          G = nx.empty_graph(n)
          for x in range(n):
              for y in range(x):
                  if random.random() < p:
                      G.add_edge(x, y)
          return G
```

```
In [18]: n = 16
          m = 15
          N = n*(n-1)//2
          m/N
```

Out[18]: 0.125

```
In [19]: G = random_graph_B(n, m/N)
         nx.draw(G, **opts)
```

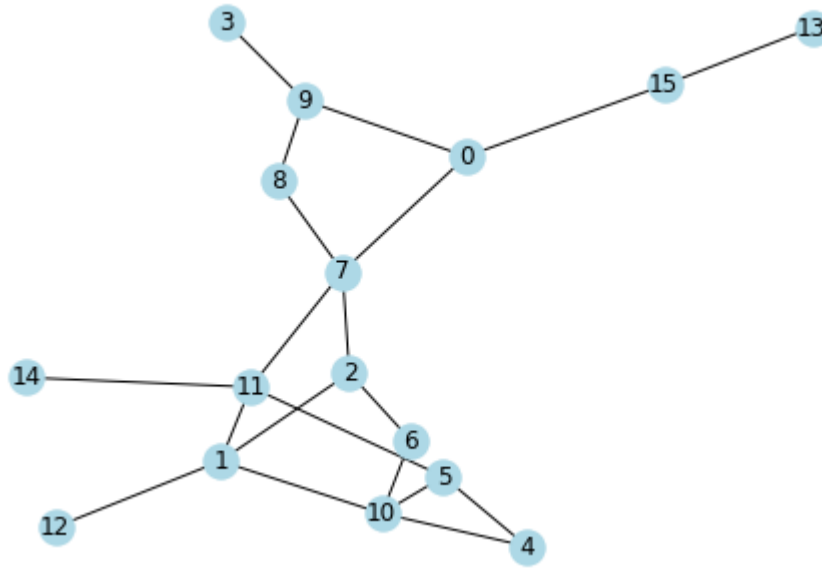


```
In [20]: G.number_of_nodes(), G.number_of_edges()
```

```
Out[20]: (16, 13)
```

The `networkx` version of this random graph constructor is called `gnp_random_graph` and should produce the same random graphs with the same probability.


```
In [21]: G = nx.gnp_random_graph(n, m/N)
         nx.draw(G, **opts1)
```



```
In [22]: G.number_of_nodes(), G.number_of_edges()
```

```
Out[22]: (16, 20)
```

Code Corner

random

- `random`: [\[doc\] \(https://docs.python.org/2/library/random.html#random.random\)](https://docs.python.org/2/library/random.html#random.random)
- `sample`: [\[doc\] \(https://docs.python.org/2/library/random.html#random.sample\)](https://docs.python.org/2/library/random.html#random.sample)

numpy

- `random.choice`: [\[doc\] \(https://docs.scipy.org/doc/numpy-1.14.0/reference/generated/numpy.random.choice.html\)](https://docs.scipy.org/doc/numpy-1.14.0/reference/generated/numpy.random.choice.html)

networkx

- `gnm_random_graph`: [\[doc\] \(https://networkx.github.io/documentation/stable/reference/generated/networkx.generators.random_graphs.gnm_random_graph.html\)](https://networkx.github.io/documentation/stable/reference/generated/networkx.generators.random_graphs.gnm_random_graph.html)
- `gnp_random_graph`: [\[doc\] \(https://networkx.github.io/documentation/stable/reference/generated/networkx.generators.random_graphs.gnp_random_graph.html\)](https://networkx.github.io/documentation/stable/reference/generated/networkx.generators.random_graphs.gnp_random_graph.html)

Exercises

1. Why does the algorithm used in the `random_sample_A` function above pick every m -element subset of the set $\{0, 1, \dots, n-1\}$ with the same probability?
2. Design an experiment, that verifies this claim. (E.g., there are $\binom{10}{3} = 120$ different 3-element subsets in a 10-element set. One could use python's `frozenset`s to make each of these 3-element sets a key in a dictionary which counts how often each set is picked when `random_sample_A` is applied a large number of times ...)
3. What is the expected number $E[m]$ of edges in the $G(n, p)$ model of random graphs?
4. What is the standard deviation σ_m of the number of edges in the $G(n, p)$ model?

In []: